NS-FirmID: A Neuro-Symbolic Multi-Agent Framework for Reliable Firmware Version Identification at Internet Scale

***Abstract*—Reliable firmware version identification is paramount for Internet-scale vulnerability assessment. Existing methods struggle with banner heterogeneity, while promising large language model solutions are hindered by deployment resource and false positive issues. We introduce NS-FirmID, the first neuro-symbolic multi-agent framework tailored for this task. By orchestrating Extraction, Verification, and Discrimination agents, our framework fuses semantic parsing with symbolic knowledge to enable robust zero-shot generalization and gray-box uncertainty modeling. Evaluations on real-world banners demonstrate that *NS-FirmID* reduces the false positive rate by over 17.1% compared to baseline SLMs while maintaining a 93% F1-score, surpassing the teacher model DeepSeek-R1 using only 1% of its parameters. Furthermore, it achieves coverage 211× broader than Nmap, revealing critical vulnerabilities in 27.4% of identified devices in large-scale assessments. Our framework is available at: https://github.com/Anonymous-for-review930/NS-FirmID.**

***Index Terms*—Firmware Identification; Neuro-Symbolic AI; Multi-Agent Framework; Large Language Models; Internet Measurement; Vulnerability Assessment.**

I. Introduction

In recent years, large-scale Internet measurement has emerged as a fundamental technique for understanding the Internet attack surface, assessing global device risk postures, and guiding patch deployment and asset management [1]. As the population of Internet-connected embedded and IoT devices continues to grow, identifying device attributes, particularly brands, models, and firmware versions, has emerged as a critical requirement for vulnerability assessment and network measurement [2]. Among these attributes, firmware version plays a uniquely important role: exploitation feasibility, patch availability, and vulnerability mappings almost always hinge on precise version information [3], [4].

Active probing remains one of the most scalable methodologies for Internet-wide device identification. By transmitting crafted packets and analyzing response banners, active probing provides immediate access to device-level information without relying on long-term traffic observation [5]. However, prior work has largely focused on coarse-grained attributes such as device type [6], brand [7], or model [8], [9]. Firmware version identification remains significantly underexplored because banners are highly heterogeneous, noisy, and often contain misleading text that mixes component versions, module identifiers, build metadata, and non-firmware software versions. Distilling the true firmware version from this noise is fundamentally challenging.

Existing efforts fall into two broad categories: implicit-feature–based fingerprinting and explicit string extraction. Methods relying on implicit features (e.g., firmware image analysis [10], DOM structure modeling [11], web directory hashing [12], [13]) can achieve high accuracy but do not scale to Internet-wide settings due to practical barriers—firmware images may be inaccessible, encrypted, proprietary, or prohibitively costly to emulate at scale. Conversely, explicit text-based approaches depend on manually maintained regular expressions or static fingerprint databases. Tools such as Nmap [14], Shodan [15] achieve high precision for a limited set of known devices, but their coverage is inherently constrained. The rapid expansion of IoT vendors, the prevalence of OEM/white-box devices, and the lack of standardized banner formats render such rule-based systems brittle, difficult to maintain, and ineffective for long-tail or previously unseen devices.

The emergence of Large Language Models (LLMs) offers a promising direction for interpreting unstructured network banners. Recent studies [2], [16], [17] have demonstrated the robust capabilities of LLMs in general device fingerprinting and label transfer. However, two fundamental barriers prevent current LLM-based approaches from reliably identifying firmware versions: 1) **The trade-off between deployment cost and model performance**: large models incur significant inference cost, limiting their practicality for high-throughput measurement. Small Language Models (SLMs), while easier to deploy, lack deep domain knowledge and struggle to extract fine-grained attributes from noisy banners [18]. 2) **Lack of structured knowledge constraints**: While LLMs facilitate flexible attribute extraction via semantic reasoning, firmware versioning strictly adheres to vendor-specific, long-standing naming conventions and sequential patterns that are difficult for purely neural models to internalize. Without explicit knowledge injection, LLMs struggle to reliably capture these symbolic structures and the interdependencies between attributes. 3) **Hallucinations and entity confusion:** LLMs face inherent reliability challenges, specifically hallucinations and entity confusion. They tend to generate false positives with high confidence, rendering them unsuitable for direct application in measurement tasks requiring high fidelity [19]

These challenges raise the central question of this work:

Can we design a system that matches or surpasses LLM-level extraction accuracy, achieves the deployment efficiency of SLMs, and provides low false positive rates

with reliable generalization to unseen firmware versions?

We present two key insights that guide our approach.

Insight 1: Firmware version identification exhibits a distinct combination of rich symbolic structure and high semantic noise. While neural models excel at parsing diverse banner formats, symbolic knowledge offers strong structural constraints and interpretability. Combining these two modalities enables a robust "extract–verify–decide" workflow that mirrors the reasoning process of human analysts

Insight 2: While prompting a model to articulate its "confidence" can induce self-reflection regarding the correctness of its answers, LLMs suffer from inherent "hallucinations" and "overconfidence, verbalized confidence only from LLMs is insufficient for security tasks. Instead, reliable uncertainty estimation must integrate intrinsic signals that cannot be fabricated during inference (e.g., token-level log-probabilities) and deterministic external evidence (e.g., semantic stability in CoT and alignment with symbolic knowledge). Such gray-box signals provide an actionable foundation for filtering hallucinations and reducing false positives.

Guided by these insights, we develop *NS-FirmID*, the first neuro-symbolic multi-agent framework for reliable firmware version identification at Internet scale. *NS-FirmID* orchestrates three complementary agents:

(1) Extractor Agent (EA): a fine-tuned Qwen2.5-7B [20] distilled from Deepseek-R1 [21], responsible for semantic parsing of banners, balances semantic reasoning with deployment efficiency.

(2) Validator Agent (VA): a symbolic component that validates extracted attributes against a comprehensive knowledge base and leverages historical naming patterns to guide the EA through iterative refinement, thereby enabling generalization to unseen versions.

(3) Discriminator Agent (DA): a gray-box uncertainty model that fuses neural, semantic, and symbolic signals into a 28-dimensional feature vector that quantifies the overall confidence of candidates, thereby suppress false positives.

NS-FirmID synergizes the neural perception of LLMs with the symbolic rigor of structured device knowledge. This hybrid approach effectively navigates the resource-precision trade-off and mitigates the hallucination-induced false positives prevalent in LLM-based firmware identification. Our contributions are summarized as follows:

- We propose *NS-FirmID*, the first neuro-symbolic multi-agent framework tailored for Internet-scale firmware identification. By orchestrating an "extract-verify-decide" workflow, it unifies semantic extraction, symbolic verification, and confidence discrimination, achieving robust structure extraction from noisy banners.
- We design a gray-box uncertainty modeling approach and a pattern induction mechanism. Besides fusing intrinsic neural signals with symbolic evidence to rigorously mitigate false positives, our pattern induction facilitates the zero-shot identification of unseen version formats
- Evaluations on 5,185 real-world banners demonstrate that *NS-FirmID* significantly outperforms SLMs in false positives (↓17.1%) and F1-score (↑14.6%), even achieves performance exceeding the teacher model while using less than 1% of its parameters.
- We executed a large-scale measurement and vulnerability assessment in the wild. Our deployment on 150,000 banners yielded firmware versions coverage surpassing Nmap by 211 times. Subsequent vulnerability mapping revealed that 27.4% of these devices are exposed to Critical-severity vulnerabilities, effectively transforming raw asset discovery into actionable risk intelligence.

RoadMap. Section II outlines the background of our work. Section III presents our fingerprint generation method. Section IV details the real-world experiments and firmware version status in the wild. Section V discusses limitations, and Section VI concludes.

II. Background

Network asset discovery is fundamental to cyberspace mapping. While methodologies generally fall into active probing or passive traffic analysis, the latter is often limited to coarse-grained classification, as distinct firmware versions typically share identical traffic signatures. Furthermore, passive models are highly sensitive to environmental noise, complicating real-world deployment. Consequently, this work prioritizes active probing, which infers precise device attributes by analyzing response characteristics to crafted packets.

Early research on device identification generally focused on coarse-grained device types [6] or brand identification [7], [22]. Yang et al. [8] utilized a Residual Dilated Gated Convolutional Neural Network to achieve model-level identification. ARE [9] further optimized the cost of extracting (Type, Brand, Model) attributes by designing an automated rule generation engine. Devtag [23] built upon ARE [9] by integrating manually crafted fingerprints and rules from existing repositories (including Nmap [14] and ZTag [24]) to achieve model-level device attribute identification.

With the rise of Large Language Models (LLMs), some studies have begun applying them to device identification. BLMProbe [2] utilized LLM-based automatic label exploration to achieve cross-port device fingerprinting, supporting both text and binary data. Sarabi et al. [16] trained a Roberta model to automatically analyze, interpret, and label large amounts of data generated by internet scans. Mahmoodden et al. [17] proposed integrating a prompt-driven LLM with an Information Theory-driven Proxy CMI to automatically generate high-confidence pseudo-labels. They further used instruction fine-tuning to address challenges like label sparsity, noise, and long-tail distributions in real-world IoT identification. However, the pseudo-labels themselves introduce noise and can amplify incidental hallucinations or erroneous associations; the explanations provided are not necessarily causally credible, and the work only focuses on manufacturer-level identification. These LLM-based efforts unanimously demonstrate the potential of LLMs in device identification and cyberspace mapping. Nevertheless, no current work has leveraged large

models to achieve fine-grained identification at the firmware version level, which further motivates our work.

Currently, research on firmware version identification in active probing scenarios is limited, mainly falling into two routes: fingerprint generation based on implicit features and keyword extraction based on plaintext. Methods based on implicit feature fingerprint generation rely on obtaining and analyzing the device's firmware binary files. They create fingerprints by extracting features that can be associated with the firmware version, suitable for scenarios where explicit information is unavailable. Methods based on plaintext keyword extraction directly extract version strings from banners or certificates obtained via active probing, making them more suitable for large-scale network measurements.

A. Fingerprint Generation Based on Implicit Features

Nmap [14], the benchmark tool in active probing, primarily uses a core mechanism of sending a series of TCP/IP packets to encode subtle differences in the target host's protocol stack implementation into a fingerprint, which is then matched against an internal operating system database using regular expressions. However, IoT devices are often based on common embedded operating system kernels. Different vendors and types of devices may run the exact same Linux kernel version. This coarse granularity of identification is insufficient for reliable vulnerability defense targeting specific device types.

Costin et al. [10] and Zhang et al. [11] used machine learning and graph neural networks to analyze internal firmware structures for version identification. Li et al. [12] generated firmware version fingerprints by comparing the file systems of firmware images. They characterized the firmware fingerprint using features derived from the hash values of the firmware's web files. Xie et al. [13] optimized the probing sequence based on Li et al.'s work [12], improving the efficiency of fingerprint identification. While precise, these methods heavily rely on the acquisition of firmware, making large-scale firmware version identification difficult because firmware may be proprietary, encrypted, or hard to emulate. Furthermore, they involve "complex feature engineering", making them unsuitable for large-scale online network measurement [1].

B. Keyword Extraction Based on Plaintext

Mainstream probing tools like Nmap [14] and ZTag [24] achieve high-accuracy firmware version identification for devices covered by their rule sets. They rely on large internal fingerprint rule bases that use hardcoded string matching or simple regular expressions. However, this approach faces a severe "long-tail challenge": the vast number of IoT device vendors, including numerous OEM (original equipment manufacturer) and unknown white-box devices, means limited rule bases cannot cover the long-tail devices.

Shodan [15] and Censys [25], as cyberspace search engines, scan Internet hosts globally using multiple protocols, significantly lowering the barrier to obtaining large-scale IoT device data. Nevertheless, they still rely on manually collected rules to identify firmware versions.

Liang et al. [26] designed efficient probing methods using reinforcement learning to find optimal multi-protocol probing strategies and extracted firmware versions from multi-port information using regular expressions. Yu et al. [**?**] attempted to log into device management interfaces using weak credentials and extracted firmware versions from locations like navigation bars using regular expressions. However, as vendors' security awareness increases, fewer devices can be accessed with weak credentials, not to mention the security risks posed by logging into the device itself. Ebbers [1] conducted a large-scale analysis of firmware version distribution in field-deployed IoT devices. This study extracted versions from the device web pages using a large number of manually designed regular expressions, demonstrating that only 2.45% of devices run the latest firmware. This work is significant in terms of measurement scale, but its methodology relies on manually designed regular expressions. Due to the inherent limitations of regular expression design, it cannot truly understand the semantic meaning of the text, often leading to low extraction success rates if the rules are too strict, or high error rates if they are too broad.

Despite the progress made by the aforementioned works, these methods severely rely on manually constructed regular expressions or predefined static fingerprint databases. They are "difficult" to maintain rules for thousands of devices and prove to be "fragile" and poorly scalable when facing the massive volume of non-standard, heterogeneous, and obfuscated banner formats prevalent in the IoT ecosystem.

III. Method

A. Problem Definition and System Overview

The goal of *NS-FirmID* is to accurately identify a device's firmware version from noisy banner strings, along with its corresponding brand and model. Given the banner text $S \in \mathcal{S}$ obtained from active probing, our goal is to extract a device attribute triplet (b, m, v), where b denotes the brand, m denotes the model, and v denotes the firmware version. Formally, we define the attribute extraction task as:

$$\mathcal{F} : S \to (b, m, v), b \in \mathcal{B}, m \in \mathcal{M}, v \in \mathcal{V}$$

where $\mathcal{B}$, $\mathcal{M}$, $\mathcal{V}$ denote the possible spaces for the brand, model, and version, respectively.

As shown in Fig. 1, *NS-FirmID* operates through three synergistic agents. The **Extractor Agent (**EA**)** utilizes a fine-tuned SLM to parse noisy banners into candidate triplets and Chain-of-Thought (CoT). These outputs are scrutinized by the **Validator Agent (**VA**)**, which leverages a symbolic knowledge base for rigorous verification and, when necessary, guides a secondary extraction via version pattern induction. To ensure high fidelity, the **Discriminator Agent (**DA**)** functions as a gray-box reliability estimator, fusing internal neural signals with external symbolic evidence into a 28-dimensional feature vector to quantify confidence.

The inference pipeline follows a progressive "Extract-Verify-Decide" workflow. Specifically, the process initiates with the EA's preliminary extraction. The VA then validates

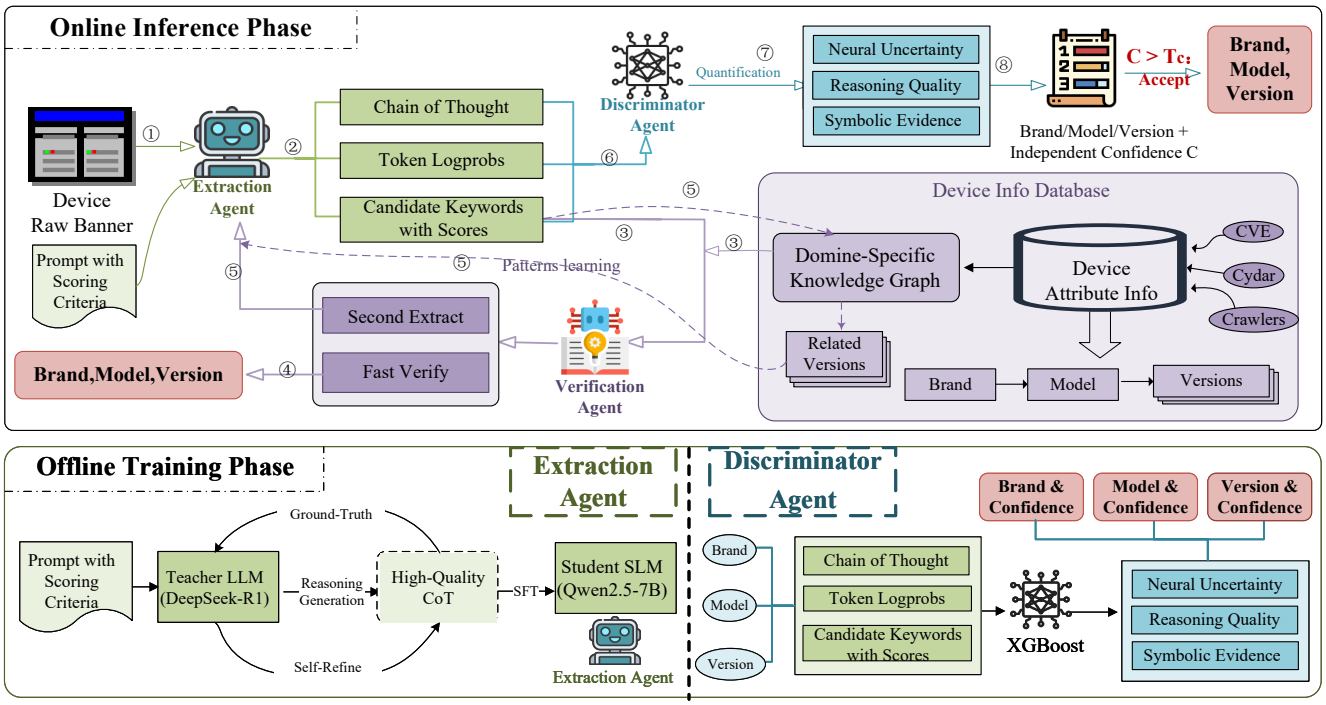

Fig. 1. The architecture of *NS-FirmID*, a neuro-symbolic multi-agent framework for reliable firmware identification from noisy Internet banners. *NS-FirmID* integrates neural extraction (EA), symbolic verification (VA), and confidence fusion (DA), leveraging structured knowledge and grey-box uncertainty reasoning to reduce false positives and generalize to unseen version formats.

these findings; if the result is incomplete or verified as uncertain, it triggers a feedback loop for knowledge-injected refinement. Finally, the DA evaluates the aggregated neuro-symbolic evidence, accepting only those attributes that satisfy a strict confidence threshold.

B. Extractor Agent

To strike a balance between deployment cost and performance, we leveraged the concept of knowledge distillation to transfer the powerful reasoning capabilities of a Teacher Model to a Student Model via Supervised Fine-Tuning (SFT). Specifically, we first guided the Teacher Model (Deepseek-R1) to generate detailed and logically clear CoT, including 'Keyword Judgment-Evidence Extraction-Confidence Scoring Details' through meticulously designed prompts. To improve the quality of CoT, we perform two rounds of generation through self-optimization of the teacher model. Then these high-quality CoT are used to fine-tune the Student SLM (Qwen2.5-7B [20]).

We use LoRa [27] for parameter-efficient fine-tuning.

To strengthen the CoT reasoning capability, we apply a higher weight to the reasoning component $\mathcal{T}$:

$$\mathcal{L}_{\text{weighted}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}[\lambda_{\mathcal{T}} \cdot \mathcal{L}_{\mathcal{T}}^{i} + \lambda_{\mathcal{E}} \cdot \mathcal{L}_{\mathcal{E}}^{i} + \lambda_{\mathcal{C}} \cdot \mathcal{L}_{\mathcal{C}}^{i}]$$

Where $\mathcal{L}_{\mathcal{T}}^{i}$ denotes the loss for the reasoning component, $\mathcal{L}_{\mathcal{E}}^{i}$ denotes the loss for the extraction result component, $\mathcal{L}_{\mathcal{C}}^{i}$ denotes the loss for the confidence analysis component. This weighting strategy encourages the model to focus more on learning the reasoning process, rather than simply memorizing the input-output mapping.

For a given banner S_i, EA yields output $\hat{y}_{EA}^{(1)}(S_i)$ with three parts:

- **Structured JSON with Confidence Scores**:
$$EA_{\mathcal{E}}^{(1)}(S_i) = \{\text{“}Brand\text{”} : \{K_{Brand}^{(1)}, C_{Brand}^{(1)}\}, \text{“}Model\text{”} : \{K_{Model}^{(1)}, C_{Model}^{(1)}\}, \text{“}Firmware\ version\text{”} : \{K_{Fw}^{(1)}, C_{Fw}^{(1)}\}\}$$
where K is the keyword of each attribute, C is the corresponding confidence score of K.
- **CoT Reasoning Text**: $EA_{\mathcal{T}}^{(1)}(S_i) = [r_b^{(1)}, r_m^{(1)}, r_v^{(1)}]$.
- **Token-Level Log Probability Sequence**: $EA_{\mathcal{C}}^{(1)}(S_i) = \{\log p_\theta(token_t|prefix)\}_{t=1}^{T}$.

C. Validator Agent

Due to the complexity and noise present in the banner, the EA may fail to identify many special version formats specific to certain brands. Furthermore, when noise in the banner confuses the version extraction, the EA's internal reasoning capability may be unable to determine whether the result is a false positive, necessitating symbolic verification via an external knowledge base.

VA maintains an IoT device attribute association graph $\mathcal{G}$ constructed from a hierarchical device Knowledge base $\mathcal{K}$ (detailed in Section 4.1):

$$\mathcal{K} = \{b : \{m : [v_1, v_2, ...]\}\}$$

where b is a brand, m is a model under brand b, and each v_i is a verified firmware version string of m.

Given the candidate prediction $EA_{\mathcal{E}}$, VA first implements a quick verification: if $EA_{\mathcal{E}}$ is a known (Brand, Model, Firmware Version) triplet with high confidence, the identification process is terminated. If the verification fails, or if the validation succeeds but yields incomplete attributes—specifically, when the EA correctly identifies a valid brand-model pair consistent with the knowledge base, but fails to extract the firmware version, a secondary extraction is triggered. If so, VA integrates the known versions related to candidate models to induce the EA to learn naming patterns and perform a knowledge-injected extraction.

Due to variations in model and version formats across different protocols and websites, we employ a hierarchical fuzzy matching algorithm in both the quick verification and the known version extraction stages. As showen in Algorithm 1, VA performs a brand → model → version progressive narrowing process to localize matching entries. First, the brand names are normalized (lowercasing, removing special characters), and then the Edit Distance similarity is calculated:

$$\text{sim}_b(b_c, b_k) = 1 - \frac{\text{EditDist}(\text{norm}(b_c), \text{norm}(b_k))}{\max(|b_c|, |b_k|)}$$

where $norm(\cdot)$ is the normalization function.

Then we conduct model matching among models belongs to the matched brand m_c. Since the model structure is richer and more prone to variations, we employ a hybird similarity combines the string similarity and semantic similarity:

$$\text{sim}_m(m_c, m_k) = \alpha \cdot \text{sim}_{\text{str}}(m_c, m_k) + \beta \cdot \text{sim}_{\text{sem}}(m_c, m_k)$$

The string similarity sim_{str} is calculated using SequenceMatcher (SM), with an extra bonus applied for substring matching:

$$\text{sim}_{\text{str}}(m_c, m_k) = \text{SM}(m_c, m_k) + \lambda \cdot \mathbf{1}[m_c \subseteq m_k \cup m_k \subseteq m_c].$$

The semantic similarity sim_{sem} is based on the structured components of the model:

$$\text{sim}_{\text{sem}}(m_c, m_k) = w_1 \cdot \delta_{\text{line}} + w_2 \cdot \text{sim}_{\text{gen}} + w_3 \cdot \text{sim}_{\text{spec}},$$

where δ_{line} is the product line matching indicator, sim_{gen} is the generation similarity, calculated as $\text{sim}_{\text{gen}} = \max(0, 1 - |g_c - g_k| \times 0.2)$, sim_{spec} is the specification matching degree (e.g., Pro, Max, Mini).

Finally, the version matching process is executed. It first attempts an exact match; if the exact match fails, fuzzy matching is subsequently employed:

$$\text{sim}_v(v_c, v_k) = \begin{cases} 1.0 & \text{if } \text{norm}(v_c) = \text{norm}(v_k) \\ \text{SM}(v_c, v_k) & \text{otherwise} \end{cases}$$

Given the matched result (b_k, m_k, v_k) and their respective similarities (s_b, s_m, s_v) , the validation confidence score is computed as a weighted sum:

$$\sigma_{\text{val}} = w_b \cdot s_b + w_m \cdot s_m + w_v \cdot s_v$$

Algorithm 1 Hierarchical Fuzzy Matching Algorithm

Require: Candidate (b_c, m_c, v_c), KB $\mathcal{K}$
Ensure: $\mathcal{V}_{results}$, $\mathcal{M}_{matches}$

$\mathcal{V}_{results}, \mathcal{M}_{matches}, \mathcal{B}_{matched} \leftarrow \emptyset$
Stage 1: Brand Matching
for $b_k \in \text{keys}(\mathcal{K})$ **do**
  $s_b \leftarrow \text{StringSim}(\text{norm}(b_c), \text{norm}(b_k))$
  if $s_b \geq 0.8$ **then** $\mathcal{B}_{matched} \leftarrow \mathcal{B}_{matched} \cup \{(b_k, s_b)\}$
  end if
end for
Stage 2: Model Matching
for $(b_k, s_b) \in \mathcal{B}_{matched}$ **do**
  for $m_k \in \text{keys}(\mathcal{K}[b_k])$ **do**
    $s_m \leftarrow \text{HybridSim}(m_c, m_k)$
    if $s_m < 0.7$ **then continue**
    end if
    $V_m \leftarrow \mathcal{K}[b_k][m_k]$; $\mathcal{M}_{matches} \cup \{(m_k, s_m, V_m)\}$
Stage 3: Version Matching
    for $v_k \in V_m$ **do**
      $s_v \leftarrow \text{VerSim}(v_c, v_k)$
      if $s_v \geq 0.8$ **then**
        $\sigma_{val} \leftarrow 0.2s_b + 0.3s_m + 0.5s_v$
        $\mathcal{V}_{results} \cup \{(b_k, m_k, v_k, \sigma_{val})\}$
      end if
    end for
  end for
end for
$\mathcal{V}_{results} \leftarrow \text{top-k}(\mathcal{V}_{results}, 5)$
$\mathcal{M}_{matches} \leftarrow \text{top-k}(\mathcal{M}_{matches}, 5)$
return $\mathcal{V}_{results}, \mathcal{M}_{matches}$

where we choose $w_b = 0.2, w_m = 0.3, w_v = 0.5$ to give higher weight to the firmware version.

When validation fails, given the model matching results $\mathcal{M}_{\text{matches}} = \{(m_k^{(i)}, s_m^{(i)}, \mathcal{V}_{m_k}^{(i)})\}_{i=1}^{K}$, we extract the version lists of the matched models as knowledge injection, combined with S_i and $\hat{y}_{EA}^{(1)}(S_i)$ for second extraction through EA. The result is denoted as $\hat{y}_{\text{VA}}^{(2)}(S_i)$, with the same structure as $\hat{y}_{\text{EA}}^{(1)}(S_i)$.

This strategy enhances the robustness of ***NS-FirmID*** when dealing with a massive volume of data from diverse sources. It is also worth noting that this approach enables the system to infer the brand inversely from the identified model, particularly in cases where the banner lacks explicit brand keywords.

D. Discriminator Agent

To mitigate the hallucination and overconfidence issues of LLM-based extraction, DA is designed to jointly model unforgeable internal reasoning signals and external symbolic evidence to quantify the uncertainty of the answer, thereby determining whether a predicted attribute is trustworthy. DA is trained independently per attribute, since their semantic spaces and difficulty differ significantly. Given the results $\hat{y}$ from EA or VA, for each attribute $\alpha \in \{brand, model, firmware\ version\}$, DA generates a structured confidence analysis

TABLE I
FEATURE SET FOR THE DISCRIMINATOR AGENT

Feature Group	Dims	Description / Components
1. Log Probability Features (8 Dims) → Neural Uncertainty Signals		
Logprob Statistics	4	$\bar{l}, \min l, \max l, \sigma_l$: Statistical measurements of log probability
Perplexity (PPL)	1	PPL: Perplexity ($\exp(-\bar{l})$)
Low-Confidence Ratio	1	r_{low}: Ratio of low-confidence tokens
Logprob Range	1	$\Delta l = \max l - \min l$: Log probability range
2. Semantic Features (5 Dims) → Reasoning Quality		
Keyword Counts	3	$n_{\text{certain}}, n_{\text{uncertain}}, n_{\text{negative}}$: Keyword counts
Reasoning Length	1	$\lvert\mathcal{T}_{\text{reasoning}}\rvert$: Length of the reasoning text
Reasoning Tokens	1	n_{token}: Number of reasoning tokens
3. Structural Features (8 Dims) → Reasoning Quality		
Has Value	1	$\mathbf{1}[\text{text}_{\text{has\_value}}]$: Whether a value has been extracted
Extracted Length	1	$\lvert v_{\text{extracted}}\rvert$: Extracted value length
Word Count	1	n_{words}: Number of words
Format Features	2	$\mathbf{1}[\text{text}_{\text{has\_number}}], \mathbf{1}[\text{text}_{\text{has\_special}}]$
Generic Term	1	$\mathbf{1}[\text{text}_{\text{is\_generic}}]$: Whether it is a generic term
Too Short	1	$\mathbf{1}[\text{text}_{\text{is\_too\_short}}]$: Whether it is too short
4. Consistency Features (3 Dims) → Symbolic Evidence		
Confidence Scores	2	$c_{\text{claimed}}, c_{\text{recommended}}$: Claimed and recommended confidence scores
Confidence Difference	1	$\Delta c = \lvert c_{\text{claimed}} - c_{\text{recommended}}\rvert$: Confidence score difference
5. Attribute-Specific Features (2 Dims) → Symbolic Evidence		
Attribute Heuristics	2	Heuristic features designed for different attribute characteristics
6. Validation Features (2 Dims) → Symbolic Evidence		
Validation Score	1	σ_{val}: Validation confidence score
Validation Success	1	$\mathbf{1}[\text{validated}]$: Whether validation passed

through a 28-dimensional feature vector $\mathbf{x} \in \mathbb{R}^d$. Features are grouped into four categories, which detailed in Table I.

DA models are trained using XGBoost [28] binary classifiers for each attribute. The output of the classifier for attribute a is defined by the following function:

$$f_a(\mathbf{x}_a) = \sigma(\mathbf{w}_a^{\mathrm{T}}\mathbf{x}_a)$$

this prediction is optimized against the ground-truth correctness label y_a for the attribute a:

$$y_a = \begin{cases} 1, & \text{LLM extraction matches ground truth} \\ 0, & \text{otherwise} \end{cases}$$

then DA models are trained by minimizing the binary cross-entropy loss across all attributes $a \in A$:

$$\mathcal{L}_{\text{DA}} = -\sum_{a \in A} \left[y_a \log f_a(\mathbf{x}_a) + (1 - y_a) \log(1 - f_a(\mathbf{x}_a)) \right]$$

IV. EVALUATION

In this section, we rigorously evaluate the performance of *NS-FirmID* using real-world Internet banner data. Our evaluation starts with the following research questions:

- **RQ1 (Effectiveness)**: Can *NS-FirmID* achieve a better balance between accuracy and recall compared to existing state-of-the-art (SOTA) firmware version identification approaches, and even match or exceed the performance of its LLM teacher model?
- **RQ2 (Recall-FPR trade-off)**: Can *NS-FirmID* mitigate false positives and hallucinations to provide trustworthy and verifiable firmware version identification while maintain high recall?
- **RQ3 (Role of Neuro-Symbolic Components)**: How do the individual agents (EA, VA, DA) contribute to the final performance? Which feature groups play the most critical role in the confidence discriminator?
- **RQ4 (Generalization and Real-World Application)**: Can *NS-FirmID* maintain robustness and scalability when deployed on large-scale, heterogeneous banners in the wild? Can the identified fine-grained versions effectively empower downstream security tasks?

A. Datasets

To evaluate *NS-FirmID* under realistic Internet-wide conditions, we construct a comprehensive dataset that integrates both active probing traces and verified firmware knowledge. Data details are listed in Table II.

Device Info Database: We build a structured knowledge base covering (brand, model, firmware version) relations by aggregating information from: 1) CPE [**?**] vulnerability database, 2) Cydar [29], an IoT asset intelligence platform, and 3) A suite of custom crawlers targeting official vendor firmware archives. This KB serves as the symbolic grounding for the subsequent validation and version pattern induction processes.

TABLE II
DETAILS OF DATASET STATISTICS

Category	Split	Samples	B	M	V	P
KG	CPE	-	1.3k	10k	41k	-
	Cydar	-	1.6k	104k	267k	-
	Crawled	-	14	16k	180k	-
	Total	-	2.7k	119k	256k	-
Labeled	Train	1925	987	533	482	51
	Test	5185	528	1545	1766	69
Unlabeled	-	150k	-	-	-	94

Note: B, M, V, P denote Brand, Model, Firmware Version, and Protocol.

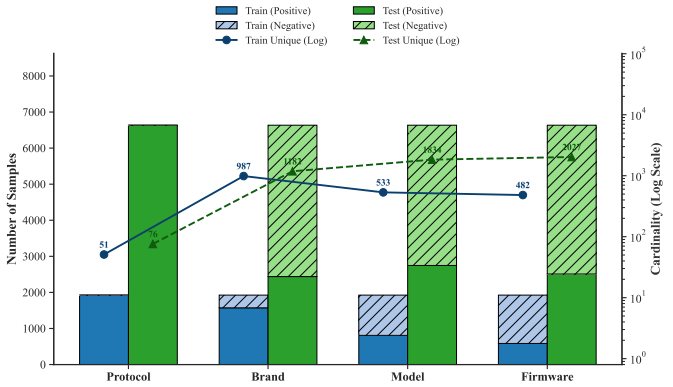

Fig. 2. Statistical comparison of the training and testing datasets across four attributes: Protocol, Brand, Model, and Firmware. The stacked bars (primary y-axis) illustrate the sample volume, segmented into positive (solid color) and negative (hatched pattern). The trend lines (secondary logarithmic y-axis) depict the cardinality (number of unique values) for each attribute.

Labeled Dataset: We actively scanned more than 100 million IPv4 hosts collected from public intelligence sources [30], [15], [31], [29]. All scans were conducted responsibly following standard ethical guidelines and respecting opt-out policies. Responses were deduplicated, initially classified via Cydar asset detection, and then manually curated by three IoT security experts. Each banner sample contains its IP address, service port, protocol type, raw banner text, and ground-truth labels (brand, model, firmware version). Due to the extremely fine granularity of firmware version identification, manual labeling at Internet scale remains highly expensive and existing automated baselines are ineffective. Consequently, the labeled dataset size is limited.

We first split the labeled data into training and testing portions at a 3:4 ratio. From the training portion, we select instances where the Teacher Model generates high-quality CoT to construct the final dataset used for SFT. The remaining data from the initial training split, along with the original testing portion, are combined to serve as the final test set, ensuring no data leakage. For the training of DA, we gathered reasoning traces from both the original Qwen2.5-7B model and the SFT-optimized Qwen2.5-7B model on the training set to ensure a diverse and comprehensive source of evidence. Fig. 2 presents a detailed statistical overview comparing the training and testing splits. As shown in the figure, the distribution of negative sample remains consistent between the training (blue bars) and testing (green bars) sets, ensuring no significant distribution shift that could bias the evaluation. To quantify the richness of our dataset, we analyze the Cardinality metric, plotted on the secondary logarithmic axis (right) to prove the high cardinality on model and firmware version, confirms the extensive coverage of our data collection.

Unlabeled Dataset: Collected in the same manner as the labeled dataset, initially filtered by Cydar but remains unverified and unlabeled. This dataset is used to assess scalability and real-world generalization.

B. Baselines

To evaluate the performance of *NS-FirmID*, we choose a set of representative baselines, including traditional rule-based methods and modern LLMs to provide a comprehensive comparison:

Nmap [14]: a well-known **open-source tool** for network exploration and security auditing, serving as a representative example of Rule-based methods. It utilizes regular expressions and keyword matching, with its service recognition rule base containing over 3, 000 rules.

Shodan [15]: a **device search engine**, scans the Internet globally across various protocols and ports. Similar to Nmap, it fundamentally relies on manually collected rules to identify specific attributes like firmware versions.

Deepseek-R1 [21]: an emerging **LLM** that demonstrates superior performance across a variety of text-related tasks. Serves as our teacher model of DA.

Qwen-14b [32], Qwen2.5-7B [20], Llama3-8b [33]: representative **SLMs** based on varying parameter sizes and distinct foundation architectures.

C. Evaluation Metrics

Following standard practice, we report Precision, Recall, F1-score, Accuracy, and False Positive Rate (FPR) to evaluate our performance:

$$\text{Precision} = \frac{TP}{TP+FP}, \quad \text{Recall} = \frac{TP}{TP+FN}$$

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{FPR} = \frac{FP}{FP+TN}, \quad \text{Accuracy} = \frac{TP+TN}{TP+FP+TN+FN}$$

where TP, FP, TN and FN refer to the true positive (the extracted value matches the ground truth), false positive (the extracted value does not match the ground truth or the ground truth is empty), true negative (the ground truth is empty, and nothing was extracted) and false negative (a value should have been extracted but was not).

TABLE III
PERFORMANCE COMPARISON OF START-OF-THE-ART METHODS

Method	Brand			Model			Firmware Version		
	F1	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall
Nmap	0.013	**0.839**	0.006	0.014	**0.886**	0.007	0.003	**1.00**	0.001
Shodan	0.166	0.312	0.112	0.036	0.060	0.025	0.062	0.606	0.033
Qwen2.5-7B	0.514	0.473	0.716	0.667	0.615	0.844	0.784	0.727	0.960
Llama-3-8B	0.508	0.381	0.760	0.621	0.551	0.712	0.660	0.579	0.768
Qwen-14B	0.480	0.320	**0.963**	0.594	0.435	**0.936**	0.670	0.507	**0.987**
NS-FirmID (7B)	**0.734**	0.713	0.924	**0.805**	0.785	0.796	**0.930**	0.927	0.887
Deepseek-R1 (671B)	0.709	0.695	0.732	0.774	0.735	0.862	0.855	0.816	0.983

D. Overall Performance (RQ1)

To evaluate the effectiveness of *NS-FirmID*, we benchmark *NS-FirmID* against four distinct categories of baseline methods IV-B. Table III presents a comprehensive comparison of all metrics across Brand, Model, and Firmware Version.

Traditional rule-based approaches exhibit severe limitations in the heterogeneous IoT ecosystem. While Nmap achieves near-perfect precision due to its rigid regular expression matching, it suffers from a catastrophic lack of recall (0.006 for firmware versions). This confirms that static signatures cannot scale to the diversity of banner formats in the wild. Conversely, standard SLMs demonstrate strong semantic understanding, achieving recalls on firmware version above 0.76, yet they suffer from hallucination and lack of domain-specific knowledge, resulting in suboptimal F1-scores. For instance, Qwen-14B, despite its larger parameter size, achieves a Recall of 0.987 on firmware version extraction but drops significantly in Precision (0.507), indicating a tendency to over-interpret noise as valid attributes. *NS-FirmID* effectively bridges this gap by synergizing neural extraction with symbolic verification. It achieves an F1-score of 0.930 on firmware version identification, surpassing the best-performing SLM baseline (Qwen-2.5-7B) by nearly 15%.

More notably, our 7B-parameter student model outperforms its 671B-parameter teacher model Deepseek-R1 while using less than **1%** of its parameters. This counter-intuitive result can be attributed to the effectiveness of our neuro-symbolic distillation and prompt design. domain-specific distillation process: while the general-purpose teacher model occasionally drifts into verbose or irrelevant reasoning, the EA has been fine-tuned to focus strictly on banner-specific patterns, and the subsequent symbolic verification phases vigorously prune the remaining errors. This counter-intuitive result can be attributed to the refined prompts designed in the domain-specific distillation process, together with the high-quality CoT generated through answer-guided and self-improving iterations. While the general-purpose teacher model may occasionally drift into verbose or irrelevant reasoning, the EA is fine-tuned to focus exclusively on banner-specific patterns, and the subsequent symbolic verification phases of VA can further optimize the results.

This demonstrates that *NS-FirmID* can overcome the inherent limitations of existing methods and small LLMs in firmware version identification, setting a new, efficient benchmark for large-scale IoT device firmware version identification. Our identified firmware version information directly empowers downstream tasks like network asset mapping and vulnerability matching, ultimately strengthening both device-level and cyberspace security.

Moreover, our framework primarily targets the previously unresolved problem of firmware version identification. Although it inherently provides brand and model extraction capabilities, it can also seamlessly integrate existing device-identification methods and treat brand and model as priors, while still maintaining reliable firmware-version inference even when the banner contains no brand or model information.

Answer to RQ1: *NS-FirmID* delivers a substantially better balance between recall and precision than both rule-based systems and SOTA LLMs, achieving teacher-level version identification at a significantly lower cost.

E. Ablation Study (RQ2 & RQ3)

To rigorously dissect the contribution of each component and evaluate its reliability calibration, we conducted a step-wise ablation study. Table IV) summarizes the performance evolution as we incrementally integrate the EA, VA, DA.

Impact of Neuro-Symbolic Components. The standard SFT (EA only) establishes a robust semantic baseline, achieving an F1-score of 0.822. However, relying solely on the SLM results in a False Positive Rate (FPR) of 14.7%, indicating that while the model learns to extract plausible strings, it struggles to distinguish between valid version numbers and stochastic hallucinations inherent to noisy banners. The integration of the VA introduces a hard symbolic constraint, rejecting candidates that violate vendor-specific formatting rules or historical naming patterns. This step yields a critical precision gain, boosting the metric from 0.779 to 0.843 and reducing the FPR to 9.6%. The most significant performance leap occurs with the inclusion of the DA. By fusing neural uncertainty with symbolic evidence, the DA suppresses the FPR to a minimal 4.1% while pushing the F1-score to 93%. Compared to the base model, our framework achieves an 17.1% reduction in

TABLE IV
ABLATION METRICS SUMMARY FOR FIRMWARE VERSION
(VALUES IN PARENTHESES INDICATE THE RELATIVE INCREASE/DECREASE COMPARED WITH THE BASE MODEL QWEN2.5-7B)

Strategy	F1	Precision	Recall	Accuracy	FPR
Base Model: Qwen2.5-7B	0.784	0.727	0.960	0.852	0.212
EA only	0.822 (↑3.80%)	0.779 (↑5.20%)	0.897 (↓6.30%)	0.869 (↑1.7%)	0.147 (↓6.50%)
EA+VA	0.871 (↑8.70%)	0.843 (↑11.6%)	0.895 (↓6.50%)	0.901 (↑4.90%)	0.096 (↓11.6%)
EA+VA+DA (*NS-FirmID*)	**0.930 (↑14.6%)**	**0.927 (↑20.0%)**	0.887 (↓7.30%)	**0.933 (↑8.10%)**	**0.041 (↓17.1%)**
Teacher Model: Deepseek-R1	0.855	0.816	**0.983**	0.897	0.169

the false positive rate and enhances the F1-score by 14.6%, albeit with a corresponding 6.5% sacrifice in recall.

Answer to RQ2: *NS-FirmID* reduces the FPR of firmware version extraction by over 17.1% compared to the Qwen2.5-7B base model, with only a marginal 6.7% sacrifice in recall. Consequently, the F1-score reaches 0.930, marking an improvement of over 14.6%, demonstrates strong resilience against hallucinations.

Reliability Calibration and Uncertainty Modeling. A core hypothesis of this work is that intrinsic neural uncertainty serves as a proxy for hallucination. We validate this via the feature importance analysis on firmware version extraction of the DA. As presented in Fig. 3, among the 28-dimensional feature set, Perplexity and Confidence Gap (the divergence between the model's claimed confidence and its token log-probabilities) emerge as dominant predictors. This confirms that when the SLM hallucinates, it exhibits measurable entropy spikes and calibration errors, which the DA effectively detects. Structural features, such as $value\_has\_special$ (presence of special characters typical in firmware versions), also play a pivotal role, proving that syntax compliance is a strong indicator of validity.

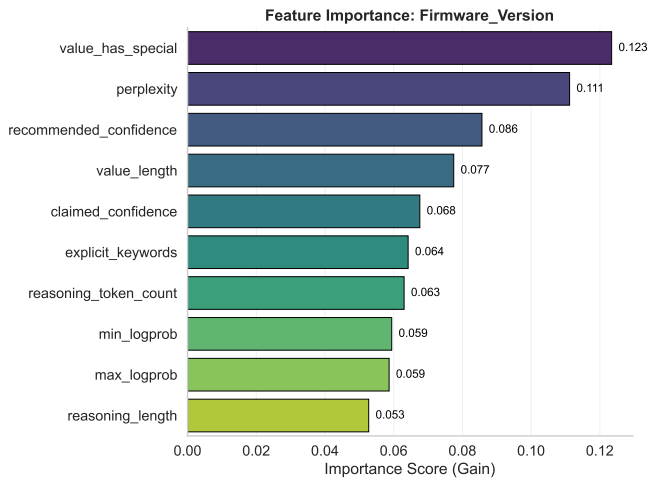

Fig. 3. Feature Importance of Firmware Version

Regularization of Reasoning. We further investigated how Domain-Specific SFT alters the model's cognitive behavior by visualizing the correlation between input banner token length and output CoT token length (Fig. 4). The base model (blue) exhibits a high-entropy distribution with scattered outliers, indicative of "hallucination loops" where the model generates excessive, circular reasoning for short, noisy inputs. This stochastic behavior not only degrades reliability but also introduces unpredictable inference latency. Conversely, the SFT-optimized EA (orange) demonstrates a linear, compact alignment. This structural regularization implies that the EA has internalized a more stable reasoning schema—"Extract, Verify, Conclude"—mitigating the stochastic over-reasoning that typically plagues SLMs in zero-shot settings.

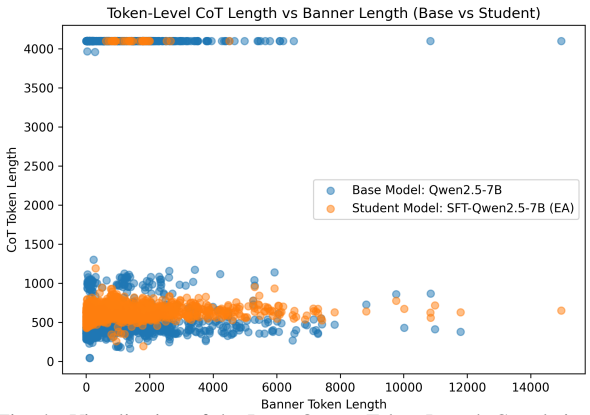

Fig. 4. Visualization of the Input-Output Token Length Correlation.

Answer to RQ3: Each agent is essential: EA provides semantic grounding, VA enforces symbolic validity and pattern induction, and DA filters false positives via the overall confidence modeling. Their synergy yields maximal F1 (0.930) and minimal FPR (0.410), confirming the necessity of all neuro-symbolic components.

F. Generalization and Application in the Wild (RQ4)

To evaluate the scalability and practical utility of our framework, we deployed *NS-FirmID* on a dataset of 150, 000 unlabeled banners collected from the public Internet and

initially filtered by Cydar. This "in-the-wild" experiment aims to assess the system's performance against the heterogeneity and noise of real-world cyberspace.

Large-Scale Asset Discovery. As illustrated in Fig. 5, *NS-FirmID* identified 129,268 devices in total, with **58,451** devices have valid firmware versions. The identification coverage is **211**× broader than Nmap (identified 277 devices with firmware versions) on the same dataset. The results reveal a highly fragmented ecosystem: we identified 25,176 distinct firmware version strings. While market leaders like Dahua and Cisco show standardized versioning (e.g., `12.4(1c)` in Cisco; `2.421.0000000.11.r` in Dahua), a long tail of small vendors employs highly heterogeneous formats (dates, commit hashes, custom codes). *NS-FirmID*'s ability to structure this chaotic data provides a high-resolution map of internet-connected devices' firmware deployment.

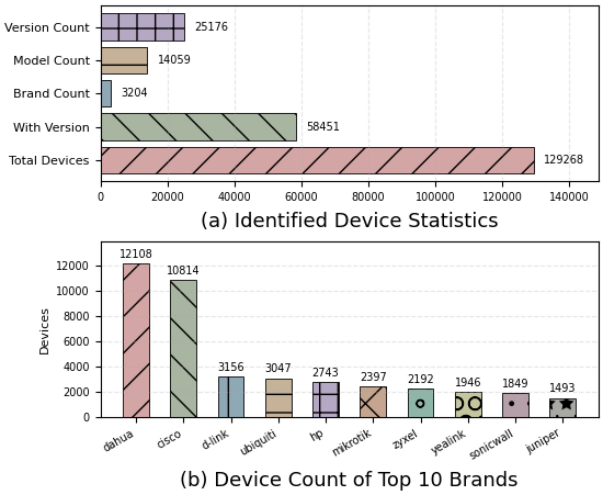

Fig. 5. Identification Results on Unlabeled Data.

Detailed Vulnerability Landscape Analysis. The primary advantage of fine-grained firmware identification lies in its ability to accurately map individual devices to the specific vulnerabilities (CVEs) associated with their firmware. To assess the vulnerability exposure of the devices we surveyed, we constructed a consolidated vulnerability repository by crawling the NVD [3]. We then matched the identified firmware versions against this database. In total, *NS-FirmID* identified 44,131 version–vulnerability correspondences. We further conducted an in-depth analysis of these matched vulnerabilities:

(1) Vulnerability Severity Distribution. As shown in Fig. 6, a substantial portion of the devices correspond to firmware versions containing serious vulnerabilities. Specifically, 27.4% of devices are linked to at least one Critical-severity CVE, and an additional 49.4% are associated with High-severity CVEs. These indicate that severe risks are far more prevalent than suggested by coarse-grained identification approaches, highlights the critical necessity of fine-grained identification methods like *NS-FirmID*.

(2) Brand-Level Vulnerability Distribution. Fig. 7 summarizes the number of matched CVEs for major vendors.

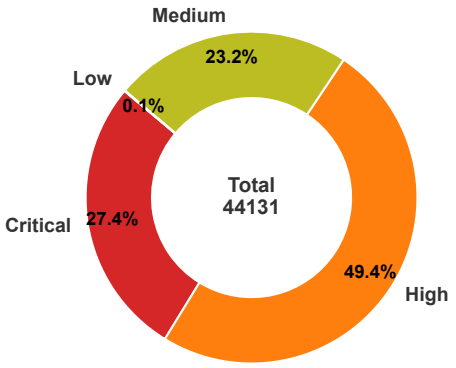

Fig. 6. Distribution of Vulnerability Severity of Identified Assets

The distribution is heavily skewed toward vendors with long product line histories and large installed bases. D-Link and Cisco together account for more than 19,000 matched CVEs, which is consistent with their broad market presence and the continued operation of older products beyond their End-of-Life cycles. Surveillance device manufacturers such as Dahua (7,302 CVEs) and Axis (3,570 CVEs) also contribute substantially. These results highlight the long-term exposure of Internet-facing surveillance systems, where firmware updates are often manual or poorly supported, leading to large numbers of unpatched devices.

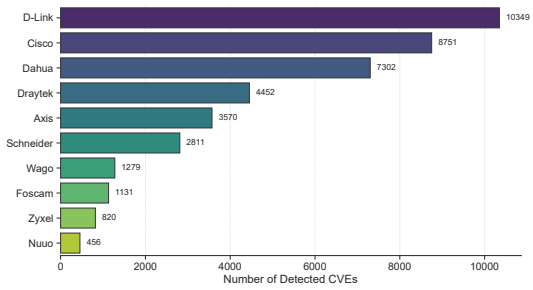

Fig. 7. Top-10 Brands with Most Vulnerabilities

(3) High-Risk Vulnerability Concentration. To focus on threats with immediate operational impact, Fig. 8 isolates only Critical and High-severity CVEs. Compared with the total-volume view in Fig. 7, a different pattern emerges: although D-Link shows the highest overall CVE count, Cisco contributes a comparable number of high-severity vulnerabilities (7,338). This indicates a higher density of severe flaws associated with Cisco's legacy devices. The persistence of high-severity CVEs across multiple vendors points to structural issues in firmware maintenance practices, underscoring the need for precise version identification such as that offered by *NS-FirmID* to distinguish genuinely vulnerable deployments from those that have been updated.

(4) Prevalence of Technical Vectors. The breakdown of CWEs in Fig. 9 sheds light on the dominant technical weaknesses associated with the matched vulnerabilities. The

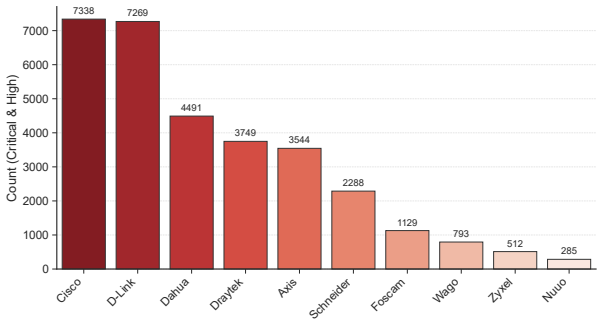

Fig. 8. Top-10 Brands with Most High-Risk Vulnerabilities

top three categories—Improper Restriction of Operations within Buffer Bounds (5,618 occurrences, e.g., CVE-2019-9676 [34]), Out-of-bounds Write (4,703 occurrences, e,g., CVE-2021-22791 [35]), and OS Command Injection (4,467 occurrences, e.g., CVE-2023-20007 [36])—all point toward long-standing issues in memory safety and command execution. These patterns are consistent with the continued reliance on C/C++ for embedded development and the limited deployment of modern mitigation mechanisms in older firmware families. The appearance of Hard-coded Credentials and Improper Authentication among the top-ranked CWEs further reflects persistent weaknesses in device access control. Unlike web applications, where vulnerabilities may cluster around client-side scripting flaws such as XSS, firmware tends to exhibit lower-level issues that can escalate to Remote Code Execution (RCE) and full device compromise.

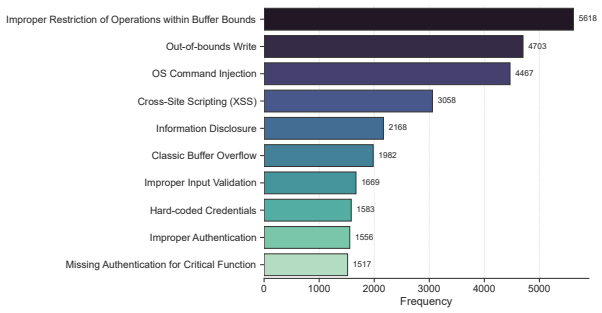

Fig. 9. Top-10 Common Vulnerability Types

The detailed results in Figs. 6–9 highlight the practical relevance of NS-FirmID. Approaches that identify only the brand or model cannot distinguish updated devices from legacy deployments, and thus obscure their actual risk profiles. By recovering precise firmware versions, our method enables direct association between individual assets and the specific CVEs affecting them. This level of precision turns general device enumeration into a more meaningful assessment of vulnerability exposure, enabling defenders to prioritize attention to the 27.4% of devices affected by critical memory corruption and injection flaws. In this sense, *NS-FirmID* helps close the gap between simply identifying devices on the network and understanding which of them present real operational risk.

Answer to RQ4: *NS-FirmID* generalizes robustly to large-scale, noisy banners. It identifies orders-of-magnitude more firmware versions than traditional tools, proving its practicality for Internet-wide measurement. Furthermore, the downstream vulnerability mapping underscores the critical value of *NS-FirmID* in cyberspace security, establishing it as a scalable solution to transform noisy network artifacts into actionable risk intelligence.

V. Limitations

Despite the significant advancements *NS-FirmID* introduces to firmware identification, we acknowledge several limitations that define the boundaries of our current approach:

Dependency on Banner Visibility: *NS-FirmID* operates on the premise that device banners contain explicit or semi-explicit version strings. Consequently, our method cannot identify devices configured with "security through obscurity" practices, such as banner suppression or complete obfuscation. This remains an inherent trade-off of lightweight active probing compared to deep, intrusive vulnerability scanning.

Inference Latency vs. Rule-Based Speed: Although the underlying SLM (7B parameters) is optimized for efficiency, the multi-agent inference process—involving CoT generation and multi-dimensional feature extraction—incurs a higher computational cost than static regex matching (e.g., Nmap). Therefore, *NS-FirmID* is best deployed as a second-stage analyzer for complex, long-tail assets where traditional rule-based tools fail, rather than a line-rate packet inspector.

Knowledge Base Completeness: The Validator Agent's effectiveness in the 'Second Pass' depends on the coverage of our knowledge base. For entirely novel vendors absent from our knowledge base, the system cannot leverage historical naming patterns and degrades to standard zero-shot extraction. While the semantic reasoning of the EA mitigates this, the reliability is naturally lower than for established vendors.

VI. Conclusion

Firmware version identification is a cornerstone of cyberspace defense, yet it has long been hindered by the heterogeneity of vendor-specific formats and the high noise inherent in network measurements. In this work, we presented **NS-FirmID**, the first neuro-symbolic multi-agent framework tailored for this challenge. By synergizing the semantic flexibility of SLMs with the rigor of symbolic verification, we established a robust "Extract-Verify-Decide" paradigm. Evaluation shows that *NS-FirmID* reduces the false positive rate by over **17.1%** compared to baseline SLMs while maintaining a F1-score of **93%**, surpassing DeepSeek-R1 using only **1%** of its parameters. In large-scale deployment, it achieved coverage **211**$\times$ broader than Nmap, the fine-grained visibility further revealed that **27.4%** of these devices are exposed to Critical-severity vulnerabilities. This work validates that Neuro-Symbolic AI offers a scalable, reliable path for the next generation of Cyberspace Mapping and Vulnerability Management.

REFERENCES

[1] Frank Ebbers, "A large-scale analysis of iot firmware version distribution in the wild," *IEEE Transactions on Software Engineering*, vol. 49, no. 2, pp. 816–830, 2022.

[2] Zhenhao Tian, Yi He, Nuo Zhang, Qixiao Lin, Hetian Shi, Jianwei Zhuge, Jian Mao, and Deliang Chang, "Blmprobe: Enhancing internet-connected device discovery by automated device labeling and label migration," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 7227–7242, 2025.

[3] "National vulnerability database," Available: https://nvd.nist.gov/, Accessed: 2025-06-12.

[4] "Common platform enumeration," Available: https://nvd.nist.gov/products/cpe, Accessed: 2025-06-12.

[5] Qiang Li, Xuan Feng, Haining Wang, Zhi Li, and Limin Sun, "Towards fine-grained fingerprinting of firmware in online embedded devices," in *2018 IEEE Conference on Computer Communications, INFOCOM 2018, Honolulu, HI, USA, April 16-19, 2018*. 2018, pp. 2537–2545, IEEE.

[6] Qiang Li, Xuan Feng, Haining Wang, and Limin Sun, "Automatically discovering surveillance devices in the cyberspace," in *Proceedings of the 8th ACM on Multimedia Systems Conference*, 2017, pp. 331–342.

[7] Xu Wang, Yucheng Wang, Xuan Feng, Hongsong Zhu, Limin Sun, and Yuchi Zou, "Iottracker: An enhanced engine for discovering internet-of-thing devices," in *2019 IEEE 20th International Symposium on" A World of Wireless, Mobile and Multimedia Networks"(WoWMoM)*. IEEE, 2019, pp. 1–9.

[8] Kai Yang, Yueyang Zhang, Xiaodong Lin, Zhi Li, and Limin Sun, "Characterizing heterogeneous internet of things devices at internet scale using semantic extraction," *IEEE Internet of Things Journal*, vol. 9, no. 7, pp. 5434–5446, 2021.

[9] Xuan Feng, Qiang Li, Haining Wang, and Limin Sun, "Acquisitional rule-based engine for discovering {Internet-of-Things} devices," in *27th USENIX security symposium (USENIX Security 18)*, 2018, pp. 327–341.

[10] Andrei Costin, Apostolis Zarras, and Aurélien Francillon, "Towards automated classification of firmware images and identification of embedded devices," in *ICT Systems Security and Privacy Protection: 32nd IFIP TC 11 International Conference, SEC 2017, Rome, Italy, May 29-31, 2017, Proceedings 32*. Springer, 2017, pp. 233–247.

[11] Weidong Zhang, Yu Chen, Hong Li, Zhi Li, and Limin Sun, "PANDORA: A scalable and efficient scheme to extract version of binaries in iot firmwares," in *2018 IEEE International Conference on Communications, ICC 2018, Kansas City, MO, USA, May 20-24, 2018*. 2018, pp. 1–6, IEEE.

[12] Qiang Li, Dawei Tan, Xin Ge, Haining Wang, Zhi Li, and Jiqiang Liu, "Understanding security risks of embedded devices through fine-grained firmware fingerprinting," *IEEE Trans. Dependable Secur. Comput.*, vol. 19, no. 6, pp. 4099–4112, 2022.

[13] Wei Xie, Chao Zhang, Pengfei Wang, Zhenhua Wang, and Qiang Yang, "Argus: assessing unpatched vulnerable devices on the internet via efficient firmware recognition," in *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security*, 2021, pp. 421–431.

[14] Gordon Fyodor Lyon, *Nmap network scanning: The official Nmap project guide to network discovery and security scanning*, Insecure, 2009.

[15] Shodan, "The search engine for internet-connected devices," Available: https://www.shodan.io/.

[16] Armin Sarabi, Tongxin Yin, and Mingyan Liu, "An llm-based framework for fingerprinting internet-connected devices," in *Proceedings of the 2023 ACM on Internet Measurement Conference*, New York, NY, USA, 2023, IMC '23, p. 478–484, Association for Computing Machinery.

[17] Rameen Mahmood, Tousif Ahmed, Sai Teja Peddinti, and Danny Yuxing Huang, "Large language models for real-world iot device identification," *arXiv preprint arXiv:2510.13817*, 2025.

[18] Kyle Mahowald, Anna A Ivanova, Idan A Blank, Nancy Kanwisher, Joshua B Tenenbaum, and Evelina Fedorenko, "Dissociating language and thought in large language models," *Trends in cognitive sciences*, vol. 28, no. 6, pp. 517–540, 2024.

[19] Zhangyue Yin, Qiushi Sun, Qipeng Guo, Jiawen Wu, Xipeng Qiu, and Xuanjing Huang, "Do large language models know what they don't know?," *arXiv preprint arXiv:2305.18153*, 2023.

[20] Qwen Team, "Qwen2.5: A party of foundation models," September 2024.

[21] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al., "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[22] Zhaoteng Yan, Zhi Li, Hong Li, Shouguo Yang, Hongsong Zhu, and Limin Sun, "Internet-scale fingerprinting the reusing and rebranding iot devices in the cyberspace," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 5, pp. 3890–3909, 2022.

[23] Shangfeng Wan, Qiang Li, Haining Wang, Hong Li, and Limi Sun, "Devtag: A benchmark for fingerprinting iot devices," *IEEE Internet of Things Journal*, vol. 10, no. 7, pp. 6388–6399, 2023.

[24] "Ztag: An utility for annotating raw scan data with additional metadata," Available: http://github.com/zmap/ztag, Accessed: 2025-11-12. [Online].

[25] Censys, "The search engine for internet-connected devices," Available: https://search.censys.io/.

[26] Chen Liang, Bo Yu, Wei Xie, Baosheng Wang, and Wei Peng, "Fine-grained identification for large-scale iot devices: A smart probe-scheduling approach based on information feedback," *Applied Sciences*, vol. 12, no. 16, 2022.

[27] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen, "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[28] Tianqi Chen, "Xgboost: A scalable tree boosting system," *Cornell University*, 2016.

[29] Cydar, "Internet of things device discovery and identification system," Available: https://cydar.cn/.

[30] Internet Assigned Numbers Authority, "Internet Assigned Numbers Authority (IANA)," Available: http://www.iana.org/.

[31] Fofa, "The search engine for internet-connected devices," Available: https://en.fofa.info/.

[32] Qwen Team, "Qwen technical report," *arXiv preprint arXiv:2309.16609*, 2023.

[33] AI@Meta, "Llama 3 model card," 2024.

[34] Common Vulnerabilities and Exposures, "CVE-2019-9676," Available: https://nvd.nist.gov/vuln/detail/CVE-2019-9676, 2019.

[35] Common Vulnerabilities and Exposures, "CVE-2021-22791," Available: https://nvd.nist.gov/vuln/detail/CVE-2021-22791, 2023.

[36] Common Vulnerabilities and Exposures, "CVE-2023-20007," Available: https://nvd.nist.gov/vuln/detail/CVE-2023-20007, 2023.